

第1章 数据类型与变量

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 建立**动态类型**世界观:变量是"名字绑定",类型属于对象而非名字
- 把 C 的 `int` / `char` / `double` 与 Python 内建类型对上号: `int` 无上限、`float` 就是 C 的 `double`
- 掌握全部字面量:进制前缀、下划线分隔符、`raw` 字符串、`f-string`、字节字面量
- 理解"常量只是约定":Python 没有 `const`,用大写命名约定
- 掌握显式类型转换与经典坑(`int("3.14")` 报错、`bool("False")` 为真)
- 会用字符串全套 API 与 `list` / `dict` / `set` 基础操作

💡 从 C 到 Python:本章主题如何迁移

在 C 里, `int x;` 意味着"在栈上分配 4 字节,类型在编译期锁死"。这就是静态类型:类型约束内存布局。Python 完全不同:执行 `x = 42` 时,Python 在**堆上**创建整数对象,让名字 `x` 指向它;再执行 `x = "hello"`,又创建字符串对象并让 `x` 指向新对象——旧对象无人引用,稍后被垃圾回收器回收。

所以 Python 的变量是**名字绑定**:名字只持有"指向对象的引用",不持有数据。类型是对象的属性,不是名字的属性。直接后果:**赋值永远只做"换标签",从不拷贝数据**;任何变量可装任何类型。这既是灵活性,也是大工程 bug 的来源——现代工程用类型标注 + `mypy` 补救(第14章)。

本章只要求两个直觉:①变量 = 名字绑定;②类型挂在对象上。两个直觉通了,后面 13 章的语法困惑会消散大半。

📦 前置知识自查(你已会的 C 基础)

- `int` / `unsigned int` / `long long` 的位数与取值范围; `sizeof`
- `char[]` 与字符串字面量、`\0` 结尾; `printf` 的 `%d` / `%s`
- `struct`、`typedef`、`const` 限定符
- 隐式转换规则(整型提升、`int`→`double`)

🚀 本章学习路线

1. **第一阶段(1 小时)**:读 §1.1–§1.2,在 REPL 里用 `type()` / `id()` 验证"名字绑定"
2. **第二阶段(1 小时)**:读 §1.3–§1.5,把 §1.5 的转换坑逐一复现
3. **第三阶段(1.5 小时)**:读 §1.6–§1.8,把 C 的字符串操作改写成 Python 版
4. **第四阶段(实验,1 小时)**:完成章末实验"学生信息登记与成绩统计"
5. **验收标准**:能说清"为什么 `int x;` 与 `x = 42` 本质不同"

本章知识导图

- §1.1 内建类型总览:与 C 的类型对照
- §1.2 变量:名字绑定与动态类型(赋值语义、id、del、命名规范)
- §1.3 字面量:整型/浮点/字符串/raw/f-string/字节/布尔/None
- §1.4 常量:约定优于强制
- §1.5 类型转换:显式转换函数与六大坑
- §1.6 字符串:不可变、切片、常用 API、编码
- §1.7 复合类型入门:list/tuple/dict/set
- §1.8 枚举与类型别名

§1.1 内建类型总览

Python 解释器自带一组内建类型,无需包含任何头文件。下表把它们与 C 类型——对照——这是本章最重要的一张表。

Python 类型	对应的 C 概念	说明与差异
<code>int</code>	<code>int / long long</code>	无上限大整数,不会溢出
<code>float</code>	<code>double</code>	就是 C 的 <code>double</code> ,没有 32 位 <code>float</code>
<code>complex</code>	—	复数,写作 <code>1+2j</code>
<code>bool</code>	<code>_Bool</code>	只有 <code>True / False</code> ,是 <code>int</code> 子类
<code>str</code>	<code>char[]</code>	Unicode 字符串,不可变
<code>bytes</code>	<code>unsigned char[]</code>	字节序列,不可变;bytearray 可变
<code>list</code>	动态数组	可变、自动扩容、元素任意类型
<code>tuple</code>	只读数组	不可变,常用于多返回值
<code>dict</code>	哈希表	键值映射,3.7+ 保持插入顺序
<code>set</code>	哈希集合	去重与集合运算
<code>NoneType</code>	<code>NULL</code> 指针	唯一实例 <code>None</code> ,表示"没有值"

★ 重点:类型是对象的属性

在 C 中"变量有类型";在 Python 中,"对象有类型,变量只是名字"。用 `type(x)` 查询 `x` 当前绑定对象的类型;同一个名字可以先后绑定不同类型的对象。`isinstance(x, T)` 判断"`x` 是否为 `T` 或其子类"——注意它在运行时检查,生产代码不要滥用。

1.1.1 用 `type()` 验证一切

拿到 Python 交互环境后第一件事:用 `type()` 验证每个字面量的真实类型。注意 `issubclass(bool, int)` 为 `True`——布尔就是整数的子类, `True + 1` 得 2,与 C 的 `_Bool` 提升一致。

示例 1-1 类型一览

```
print(type(42))           # <class 'int'>
print(type(3.14))         # <class 'float'>  — 就是 double!
print(type(True))         # <class 'bool'>
print(type("hi"))         # <class 'str'>
print(type(b"hi"))        # <class 'bytes'>
print(type(None))         # <class 'NoneType'>
print(type([1, 2]))       # <class 'list'>
print(type((1, 2)))       # <class 'tuple'>
print(type({"k": 1}))     # <class 'dict'>
print(type({1, 2}))       # <class 'set'>
print(issubclass(bool, int)) # True
print(True + 1)           # 2
```

1.1.2 int 不会溢出

C 的 `int` 加 1 就溢出(有符号溢出还是 UB);Python 的 `int` 是任意精度的, `10**100` 得到完整精确的整数。代价:每个整数都是堆上对象,运算比机器整数慢一个数量级——这就是动态类型的取舍。整数写法:二进制 `0b1010`、八进制 `0o12`、十六进制 `0xFF`;没有 `unsigned`,因为大整数天然无符号概念。

§1.2 变量:名字绑定与动态类型

C 的声明与初始化是两件事,且未初始化局部变量是 UB。Python 只有一种动作: `x = 5` ——不存在“声明”,赋值即创建。读一个从未赋值的名字抛 `NameError`,而不是像 C 一样给你随机值——错误被显式暴露,这正是 Python 好调试的原因。

C 的写法	Python 的写法
<code>int x = 5;</code>	<code>x = 5 # 赋值即"出生"</code>
<code>int y; /* 未初始化→UB */</code>	<code># 不赋值就没有这个名字</code>
<code>x = 3.14; /* 编译错误 */</code>	<code>x = 3.14 # 合法:重新绑定</code>
<code>printf("%d", x);</code>	<code>print(x) # 类型随对象走</code>

1.2.1 赋值语义:绑定引用,不拷贝数据

执行 `b = a` 时,C 拷贝值;Python 拷贝“引用”。若对象可变(如 `list`),两个名字共享同一对象,修改其一,另一个也变——这是 C 程序员踩的第一个 Python 坑。想拷贝用切片 `a[:]` 或 `list(a)` (浅拷贝,深拷贝见第14章)。生产中最常见的例子是函数传参:把列表传给函数,函数里 `append`,调用方的列表也跟着变——第4章会系统讲透这个语义。

示例 1-2 名字绑定 vs 拷贝

```

a = [1, 2, 3]
b = a                # 不拷贝!b 与 a 指向同一个对象
b.append(4)
print(a)              # [1, 2, 3, 4] — a 也"变"了

print(id(a) == id(b)) # True — id 是对象标识(类比地址)

c = a[:]              # 浅拷贝:新列表
c.append(5)
print(a)              # [1, 2, 3, 4] — a 不受影响

# 不可变对象没有这个问题:任何"修改"都产生新对象
s = "hi"
t = s + "!"          # 创建新字符串,t 重新绑定
print(s)              # hi(原对象未动)

```

⚠ 误区 1:以为 `b = a` 之后两者独立

C 里 `int b = a;` 后两者互不影响;Python 里 `b = a` 只是把 `b` 绑到 `a` 的对象上。对**可变对象**(list/dict/set/自定义对象), `b.append(...)` 会反映到 `a`;对**不可变对象**(int/float/str/tuple)则安全。记不住就问自己:这个对象能 `append` 吗?能,就共享了。

1.2.2 命名规范与 `del`

PEP 8 约定:变量与函数用 `snake_case`,类名 `PascalCase`,常量 `ALL_CAPS`。这些是规范而非语法,但违反规范 = 制造混乱。`del x` 解除名字绑定(不是 `free`,对象等垃圾回收);`del lst[0]` 删元素;`del d["k"]` 删键值对。

§1.3 字面量

字面量是"直接写在源码里的值"。Python 的字面量家族比 C 丰富得多,一个示例全部覆盖:

示例 1-4 字面量全家桶

```

# ---- 整数 ----
n1 = 42                # 十进制
n2 = 0x2A              # 十六进制(与 C 一致)
n3 = 0o52              # 八进制(C 写作 052)
n4 = 0b101010         # 二进制(C 无原生支持)
n5 = 1_000_000         # 下划线分隔符(3.6+, 可读性利器)
print(n2, n3, n4, n5)  # 42 42 42 1000000

# ---- 浮点 ----
f1 = 3.14
f2 = 1.5e-3            # 科学计数法
f3 = float("inf")      # 正无穷(还有 -inf 与 nan)

# ---- 字符串 ----
s1 = "单引号也行"      # 两种引号等价
s2 = ""可以跨行""      # 三引号:多行字符串
s3 = r"C:\Users\new"   # raw 字符串:反斜杠不转义!
s4 = b"byte"           # 字节字面量,类型 bytes
s5 = f"1+1={1+1}"      # f-string:花括号内是表达式
s6 = f"{s1!r}"         # !r 用 repr 显示(带引号)

# ---- 特殊值 ----
t = True               # 布尔只有 True / False
z = None               # None:类比 NULL 指针

```

⚠ 误区 2:反斜杠与 C 的记忆冲突

在 C 里 "C:\new" 的 \n 是换行;Python 同样如此!所以 Windows 路径必须写 r"C:\new" (raw)或 "C:\\new" 或 "C:/new"。生产代码里写 Windows 路径,永远用 raw 字符串或 pathlib(第7章)。

1.3.1 f-string:现代格式化主力

C 用 `printf("%d, %s", n, s)` 占位;Python 3.6+ 首选 f-string:字符串前加 `f`,花括号里直接写表达式。旧式 `%` 格式化与 `str.format()` 只在兼容老代码时用。

示例 1-5 f-string 实战

```

name, score = "李雷", 92.5
print(f"{name} 的成绩是 {score:.1f} 分")  # 李雷 的成绩是 92.5 分

print(f"{name:<10}|")    # 左对齐占 10 格
print(f"{name:>10}|")    # 右对齐占 10 格
print(f"{score:06.1f}")  # 总宽 6 补零 1 位小数 → 0092.5

width, height = 3, 4
print(f"面积 = {width * height}")  # 花括号里能算
print(f"数名:{name!r}")           # 调试神器:带引号

```

§1.4 常量:约定优于强制

Python **没有** `const` / `#define`。社区约定:全大写命名,并约定"不修改"——解释器不拦你。这是刻意设计:Python 认为"约束靠自觉与审查,而非编译器"。防修改的三种手段:

- 用不可变对象: `tuple`、`frozenset` 无法就地修改
- 类型标注 `typing.Final` + `mypy` 静态检查(第14章)
- 模块级集中定义:外部 `import config` 只读使用

示例 1-6 常量的正确打开方式

```
# config.py — 常量集中管理,别散落各处
MAX_RETRY = 3
TIMEOUT_SEC = 30.0
DEFAULT_ENCODING = "utf-8"
SUPPORTED_EXT = frozenset({".py", ".md", ".txt"}) # 防误改

def load_file(path: str):
    if not path.endswith(tuple(SUPPORTED_EXT)):
        raise ValueError(f"不支持的扩展名:{path}")
    return open(path, encoding=DEFAULT_ENCODING)

# 可变 vs 不可变
a = (1, 2) # tuple 不可变
# a[0] = 9 # TypeError: 'tuple' object does not support item assignment
b = [1, 2] # list 可变
b[0] = 9 # 合法
```

§1.5 类型转换

C 有隐式转换(整型提升、`int`→`double`)和显式转换 `(double)x`。Python 的立场:数值间保留少量隐式转换,其余一律显式。隐式转换只发生在 `int` ↔ `float` ↔ `complex` 的算术里;`int` 与 `str` 之间**绝不**隐式转换——`"a" + 1` 直接抛 `TypeError`,而不是像 JS 那样偷偷变成 `"a1"`。这就是"强类型动态语言"。

C 的写法	Python 的写法
<code>(double)x / (int)y</code>	<code>float(x) / int(y)</code>
<code>sprintf(buf, "%d", n)</code>	<code>f"{n}"</code> 或 <code>str(n)</code>
<code>atoi("42")</code>	<code>int("42")</code> (非数字抛 <code>ValueError</code>)
字符 ↔ 数字	<code>ord("A")</code> → 65; <code>chr(65)</code> → "A"

1.5.1 转换的六大经典坑

示例 1-7 转换的坑与正确姿势

```
# 坑 1:int("3.14") 失败!int() 不接受小数点字符串
# n = int("3.14")          # ValueError: invalid literal
n = int(float("3.14"))     # 正确姿势:先 float 再截断 → 3

# 坑 2:int("42.0") 同样失败——先浮点再整
# int("42.0")              # ValueError

# 坑 3:进制字符串要带第二参
v = int("ff", 16)          # 255
v2 = int("1010", 2)        # 10

# 坑 4:bool("False") 是 True!非空字符串恒为真
print(bool("False"), bool(""), bool(0)) # True False False

# 坑 5:float 截断是向零截断,不是四舍五入
print(int(3.7), int(-3.7)) # 3 -3

# 坑 6:首尾空格被容忍,中间的不行
print(int(" 42 "))        # 42
# int("4 2")              # ValueError
```

⚠ 误区 3:以为 Python 会自动截断或拼接

7 / 2 得 3.5 (真除法,不截断!);要截断用 7 // 2。"1" + "2" 是 "12" 不是 3;数字与字符串拼接必须 f"{n}"。一切"自动"都没有,一切显式——这正是 Python 好调试的原因。

§1.6 字符串:从 char 数组到一等公民

C 的字符串是 `char[]`,拼剪全靠 `strcpy / strcat`,还提防缓冲区溢出。Python 的 `str` 是**不可变的 Unicode 字符串序列**:没有 `\0`、没有缓冲区边界。不可变带来直接好处:字符串对象可以被安全共享,不用担心某处修改破坏别处。

1.6.1 索引、切片与常用 API

索引从 0 开始,负数从末尾数(`s[-1]` 是末字符)。切片 `s[start:stop:step]` 是 Python 最强特性,口诀:左闭右开,负数倒数,步长可负。

示例 1-8 字符串 API 速览

```
s = "Hello, Python"
print(s[0], s[-1])          # H n
print(s[0:5])               # Hello(不含下标 5)
print(s[7:])                 # Python(到末尾)
print(s[::-1])               # nohtyP ,olleH(逆序!)

print(s.find("Py"))          # 7;找不到返回 -1 不抛异常
print(s.replace("Python", "C")) # Hello, C
print("a,b,c".split(","))    # ['a', 'b', 'c']
print("-".join(["2026", "08", "26"])) # join 在分隔符上!
print(" hi ".strip())        # hi
print("abc123".isalpha())    # False

print(len("你好"))           # 2 — len 是字符数,不是字节数
```

1.6.2 编码:str 与 bytes 的鸿沟

C 的 `char` 是字节;Python 的 `str` 是 Unicode 码点。读写文件、发网络包时, `str` 必须编码成 `bytes`, 反之解码。`bytes` 只是字节,怎么解释成字符由编码决定;现代统一用 UTF-8。

示例 1-9 编码与解码

```
text = "中文Python"
data = text.encode("utf-8")    # str → bytes
print(data)                   # b'\xe4\xb8\xad\xe6\x96\x87Python'
print(len(text), len(data))   # 8 字符 vs 13 字节

back = data.decode("utf-8")    # bytes → str
print(back == text)           # True

# 编码不一致会乱码或报错;文件读写统一
# 指定 encoding="utf-8"(第7章详述),别依赖系统默认
```

§1.7 复合类型入门:list / tuple / dict / set

C 数组定长同型;Python 的 `list` 是变长、任意元素、自动扩容的动态数组——C 里手写 `realloc` 数组的成品版。`list` 的索引与切片规则和字符串完全一致:负数下标从尾部数起,切片左闭右开。这里给基础操作,第14章从内存视角深入。

示例 1-10 list/dict/set 基础操作

```
# ---- list ----
nums = [3, 1, 4, 1, 5]
nums.append(9)    # 尾部追加(自动扩容)
nums.insert(0, 0) # 头部插入
nums.sort()       # 就地排序
print(nums)       # [0, 1, 1, 3, 4, 5, 9]
print(len(nums))  # 7

# ---- tuple:不可变,常用于打包 ----
point = (3, 4)
x, y = point      # 解包
single = (1,)     # 单元元素组必须带逗号!

# ---- dict ----
student = {"name": "韩梅梅", "age": 18, "scores": [88, 92, 95]}
print(student["name"])    # 韩梅梅;键不存在抛 KeyError
print(student.get("grade", "N/A")) # N/A:get 带默认值
student["grade"] = "大一"  # 新增键值对
for k, v in student.items(): # 遍历键值
    print(k, v)

# ---- set:去重 ----
tags = {"python", "c", "python"}
print(tags)    # {'c', 'python'}
print("c" in tags) # True(哈希查找)
```


⚠ 误区 4:dict 取键以为像数组越界一样安全

`d["k"]` 键不存在抛 `KeyError` (会显式报错,这是好事)。生产习惯:不确定键是否存在用 `d.get(key, default)`;遍历用 `for k, v in d.items()`。另外 dict 的键必须可哈希——list 不能做键,tuple 可以,因为 dict 底层就是哈希表。

§1.8 枚举与类型别名

C 的 `enum` 只是带名字的整数,可以把 42 直接赋给枚举变量,零检查。Python 的 `enum.Enum` 是真正类型:成员是对象,值必须合法,repr 清晰,还能带方法。`IntEnum` 可当整数无缝使用(如做下标)。

示例 1-11 枚举与类型别名

```
from enum import Enum, IntEnum

class Color(Enum):
    RED = 1
    GREEN = 2
    BLUE = 3

class Level(IntEnum):          # IntEnum 可以当 int 用
    LOW = 1
    MEDIUM = 2
    HIGH = 3

print(Color.RED.name, Color.RED.value)    # RED 1
print(Color(2))                          # Color.GREEN:按值反查
print(Level.HIGH > Level.LOW)             # True

# 类型别名:C 的 typedef 在 Python 里就是普通赋值
StudentId = int
sid: StudentId = 20260001                # 标注只是说明,不强制
print(type(sid))                         # <class 'int'> — 别名不产生新类型
```

🚀 工程建议:枚举优先于魔法数字

生产代码里,状态、等级、类型字段一律用 `Enum` 而非裸整数:`order.status = OrderStatus.PAID` 比 `order.status = 3` 可读得多,还能防越界值。这是"可读性投资"里回报率最高的一项;`StrEnum` (3.11+)可直接当字符串用。

本章速查表

主题	写法 / 结论
类型查询	<code>type(x)</code> ; <code>isinstance(x, T)</code>
整数	任意精度; <code>0x</code> / <code>0o</code> / <code>0b</code> 前缀; <code>1_000_000</code>
浮点	只有 <code>double</code> ; <code>1.5e-3</code> 、 <code>float("inf")</code>
None	判空写 <code>x is None</code> 而非 <code>==</code>
变量	赋值即创建; 名字可重新绑定; 无未初始化
拷贝	<code>b = a</code> 只绑引用; <code>a[:]</code> 浅拷贝
常量	无 <code>const</code> ; ALL_CAPS + 集中定义 + 自觉
字符串	不可变; 负索引; <code>s[1:4]</code> 左闭右开; <code>len</code> 数字符
f-string	<code>f"{x:.2f}"</code> 、 <code>f"{x:>10}"</code> 、 <code>f"{x!r}"</code>
转换	<code>int(x)</code> / <code>float(x)</code> / <code>str(x)</code> ; <code>int(s, 16)</code>
编码	<code>s.encode("utf-8")</code> ↔ <code>b.decode("utf-8")</code>
list/dict	<code>append</code> / <code>sort</code> ; <code>d.get(k, 默认)</code> 安全
枚举	<code>Enum</code> 类型安全; <code>IntEnum</code> 当 <code>int</code> 用

最小必会清单

- 能说出: 变量是"名字绑定", 类型属于对象, 赋值不拷贝数据
- 能写出 `int/float/bool/str/bytes/None/list/dict/set` 各一个字面量
- 能用 `type()/id()` 验证"同一个对象"与"同一个值"的区别
- 能解释 `int("3.14")` 为什么报错、正确写法是什么
- 能写出 `f"{name:>10}"` 右对齐与 `f"{x:.2f}"` 保留小数
- 能说出切片左闭右开规则, 写出逆序切片 `s[::-1]`
- 能区分 `str`(字符)与 `bytes`(字节), 知道 `encode/decode`
- 能用 `d.get(key, default)` 安全读字典
- 能解释 `bool("False")` 为什么是 `True`

自测题

Q 1. 执行 `a = [1, 2]; b = a; b.append(3)` 后 `a` 是什么? 若 `a = "hi"; b = a; b = b + "!"`, `a` 呢? 为什么不同?

✅ 参考答案

`[1, 2, 3]`: list 可变, `b = a` 共享对象。字符串场景 `a` 仍是 `"hi"`: str 不可变, `b + "!"` 创建新对象让 `b` 重新绑定。核心: 可变对象共享修改, 不可变对象"修改"即新建。

Q 2. 下列哪个抛异常? ① `int("3.14")` ② `float("3.14")` ③ `int("0xff", 16)` ④ `int("1e3")` ⑤ `bool("False")`

✓ 参考答案

① 和 ④ 抛 `ValueError: int()` 不接受小数点与指数记法;② 得 3.14;③ 得 255;⑤ 得 True。

Q 3. `s = "Hello, World"` ,写出取 `"World"` 的切片、逆序切片、最后一个字符的表达式。

✓ 参考答案

`s[7:]` (或 `s[-5:]`)、`s[::-1]`、`s[-1]`。切片左闭右开,省略 stop 到末尾。

Q 4. `7 / 2`、`7 // 2`、`-7 // 2`、`-7 % 2` 各是多少?与 C 有何不同?

✓ 参考答案

3.5、3、-4、1。Python 的 `//` 与 `%` 向下取整(符号随除数),恒有 `a == (a//b)*b + a%b`;C 向零截断(`-7/2 == -3`)。

Q 5. 为什么 `len("中文")` 是 2 而 `len("中文".encode("utf-8"))` 是 6?这说明了什么?

✓ 参考答案

str 的 len 数 Unicode 字符(码点),bytes 的 len 数字节。中文在 UTF-8 中每字 3 字节,2 字符 = 6 字节。str 面向人类可读字符,bytes 面向存储传输,必须显式编码/解码。

Q 6. `d = {}` ,依次执行 `d["a"] = 1`、`d["b"] = 2`、`d.setdefault("a", 100)`、`d["a"] = d.get("a", 0) + 1` ,最终 d 是什么?

✓ 参考答案

`{'a': 2, 'b': 2}`。setdefault 只在键不存在时写入, "a" 保持 1; `d.get("a", 0)` 得 1,加 1 回写为 2。这类场景 `collections.defaultdict` 是更优解。

章末实验题:学生信息登记与成绩统计系统

实验 1:学生信息登记与成绩统计

实验目标:用全章所有数据类型(整型/浮点/布尔/字符串/list/dict/枚举/常量/转换/格式化)实现一个命令行学生管理系统。

实验要求:

- 任务 1:用 `Enum` 定义年级枚举(FRESHMAN/SOPHOMORE/JUNIOR/SENIOR)(知识点:枚举)
- 任务 2:用常量模块定义配置:最大学生数、及格线 60.0、编码 utf-8(知识点:常量约定)
- 任务 3:单个学生用 dict 存储{学号、姓名、年龄、年级、是否在校 bool、成绩 list},总表用 list(知识点:复合类型)
- 任务 4:录入时用 `input()` 读字符串, `int()` / `float()` 转换并捕获 `ValueError` 重输(知识点:类型转换与异常)
- 任务 5:统计平均分、最高分、及格率,f-string 格式化输出(知识点:格式化与算术)
- 任务 6:按学号查找学生并打印完整信息(知识点:dict 查询)

实验环境:Python 3.10+,标准库即可。

第一步 写 `config.py` 与 `student.py` (常量与枚举),再写主程序,保持"数据定义在前、逻辑在后"

第二步 入函数用 `while True + try/except` 循环处理非法输入——生产最常见的输入校验模式

第三步 计用 `sum()/max()` 或手动循环;平均分用真除法 `/` 得到 `float`

第四步 出报告用 `f"{avg:.1f}"` 控制小数位

✅ 参考答案要点(代码分两段)

第一段:配置、枚举与录入。关键设计:枚举隔离魔法数字、常量集中配置、录入函数返回形状统一的 dict。

```
# config.py — 常量集中定义
MAX_STUDENTS = 100
PASS_LINE = 60.0
ENCODING = "utf-8"

# student.py — 枚举定义
from enum import Enum

class Grade(Enum):
    FRESHMAN = 1      # 大一
    SOPHOMORE = 2     # 大二
    JUNIOR = 3        # 大三
    SENIOR = 4         # 大四

# main.py — 主程序
from config import MAX_STUDENTS, PASS_LINE, ENCODING
from student import Grade

def make_student(sid: str, name: str, age: int,
                 grade: Grade, active: bool, scores: list) -> dict:
    """构造一个学生 dict(形状统一,便于后续处理)"""
    return {
        "sid": sid, "name": name, "age": age,
        "grade": grade, "active": active, "scores": scores,
    }

def input_score(prompt: str) -> float:
    """读入成绩,非法输入反复要求重输"""
    while True:
        raw = input(prompt).strip()
        try:
            value = float(raw)
            if 0.0 <= value <= 100.0:
                return value
            print("成绩必须在 0~100 之间")
        except ValueError:
            print("请输入数字")
```

第二段:录入组装、统计报告与主流程。注意学号查找用了 dict 存储,查找 O(1)。

```

def input_student() -> dict:
    """录入一个学生:覆盖类型转换、枚举反查"""
    sid = input("学号: ").strip()
    name = input("姓名: ").strip()
    age = int(input("年龄: "))          # 非法整数抛 ValueError
    grade_no = int(input("年级(1~4): "))
    grade = Grade(grade_no)           # 按值反查枚举成员
    active = input("是否在校(y/n): ").strip().lower() == "y"
    scores = [input_score(f"第{i}门成绩: ") for i in range(1, 4)]
    return make_student(sid, name, age, grade, active, scores)

def report(students: list) -> None:
    """统计并格式化输出"""
    total = [sum(s["scores"]) / len(s["scores"]) for s in students]
    avg = sum(total) / len(total)
    top = max(total)
    rate = sum(1 for t in total if t >= PASS_LINE) / len(total) * 100
    print("=" * 40)
    print(f"学生人数: {len(students)} | 平均: {avg:.1f} | 最高: {top:.1f}")
    print(f"及格率: {rate:.1f}%")
    print("=" * 40)
    for s in students:
        avg_s = sum(s["scores"]) / len(s["scores"])
        tag = "及格" if avg_s >= PASS_LINE else "不及格"
        print(f"{s['sid']} {s['name']} 年级={s['grade'].name} "
              f"平均={avg_s:.1f} [{tag}]")

def find_student(students: list, sid: str) -> dict | None:
    """按学号查找;找不到返回 None(True 值判定)"""
    for s in students:
        if s["sid"] == sid:
            return s
    return None

def main() -> None:
    students = []
    while input("继续录入? [y/n]: ").strip().lower() == "y":
        try:
            students.append(input_student())
        except (ValueError, TypeError) as exc:
            print(f"录入失败,已跳过:{exc}")
    report(students)
    sid = input("查询学号: ").strip()
    stu = find_student(students, sid)
    if stu is not None:          # 判 None 用 is
        print(f"{stu['name']} 平均分 "
              f"{sum(stu['scores']) / len(stu['scores']):.1f}")
    else:
        print("未找到该学号")
    with open("students.txt", "w", encoding=ENCODING) as fh:
        for s in students:
            fh.write(f"{s['sid']},{s['name']},{s['age']},")

```

```
f"{s['grade'].value},{int(s['active'])}\n")
```

```
if __name__ == "__main__":  
    main()
```

设计说明:① 录入函数把"读字符串→校验→转类型→组装"拆成独立步骤,任何一步失败都能单独测试;② 枚举成员用 `.name` 显示、`.value` 持久化,体现"面向人的表示"与"面向机器的表示"分离;③ 保存文件用 `with open` 保证资源释放(第5、7章细讲),显式 `encoding="utf-8"` 避免 Windows 默认 GBK 乱码——这条在真实生产里救过无数人。

下一章预告:第2章 运算符与表达式——`//` 向下取整、`**` 幂运算、短路返回操作数、链式比较、`walrus` 操作符,Python 的运算符世界与 C 有 8 处本质不同。